NOVA IMS INFORMATION MANAGEMENT SCHOOL

Three ways to be wrong

Computation I · Lecture 01

Prof. Karina Brotto Rebuli
krebuli@novaims.unl.pt

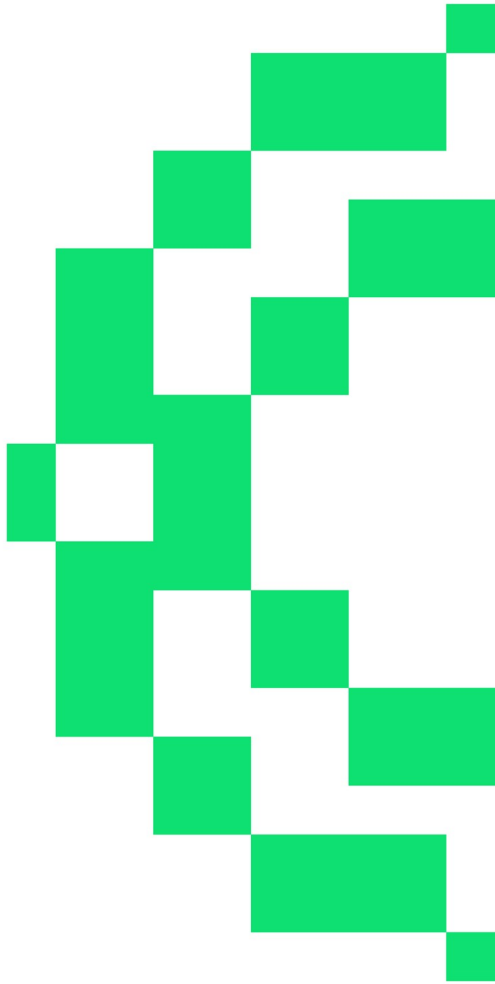
“Go to the store and buy a loaf of bread.

If they have eggs, buy six.”

A wife, to her husband, who is a very good programmer.

He came home with six loaves of bread.

“Why six loaves of bread???”
… “Because they had eggs.”

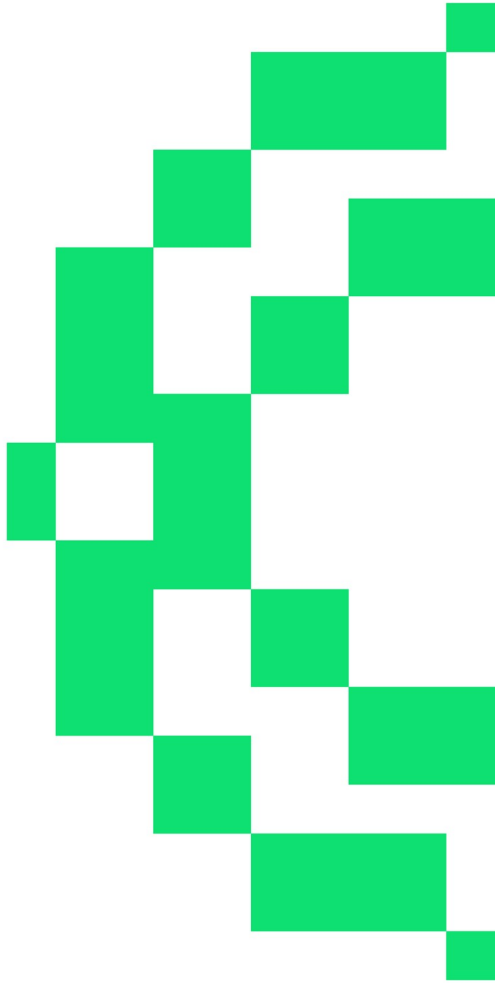


He wasn't stupid. He was precise.

He did exactly what was said, not what was meant.

“Go to the store and buy a loaf of bread.

If they have eggs, buy six.”



This program calculates your final average

It runs. Nothing goes wrong.

```
Grade 1: 19
Grade 2: 11
Grade 3: 8
Grade 4: 6

Final average: 6.25
```

Your grades were 19, 11, 8 and 6

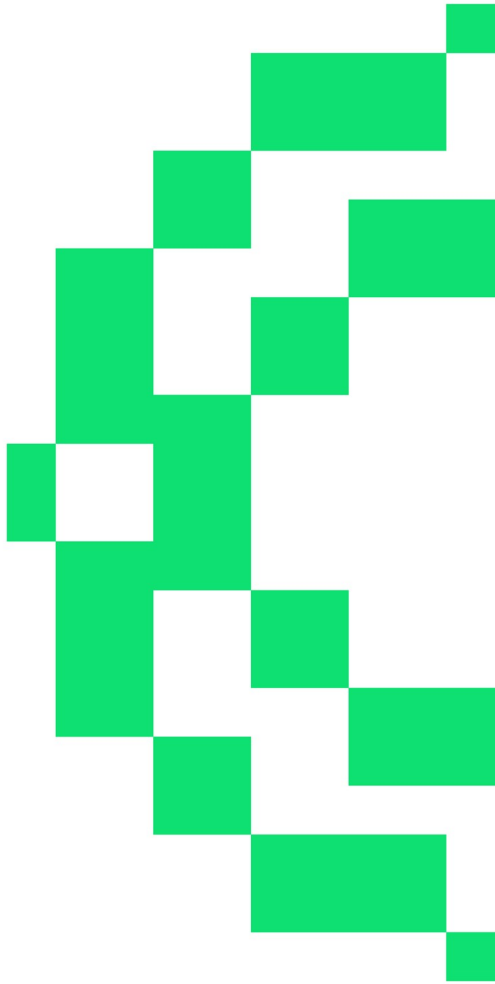
Your average should be 11.0

6.25 `is not` **11.0**

(11 + 8 + 6)/4 = 6.25

The program quietly dropped your best grade, and failed you for it.

```
"""Code that runs is not code that's right."""
```





Computer History Museum

Grace Hopper

Harvard, 1947 — the machine stopped, and nobody knew why

The problem: the machine does what you told it, at a speed no human can follow, and gives you no account of what it did. So when there is a fault, or a bug, how do you find out where?

Three questions today

What actually happens when you run a program.

What the machine tells you when you are wrong.

What it doesn't tell you.

01.

What actually happens when you run a program

Between pressing Run and seeing the answer

LIVE

Your first program — Hello, world!

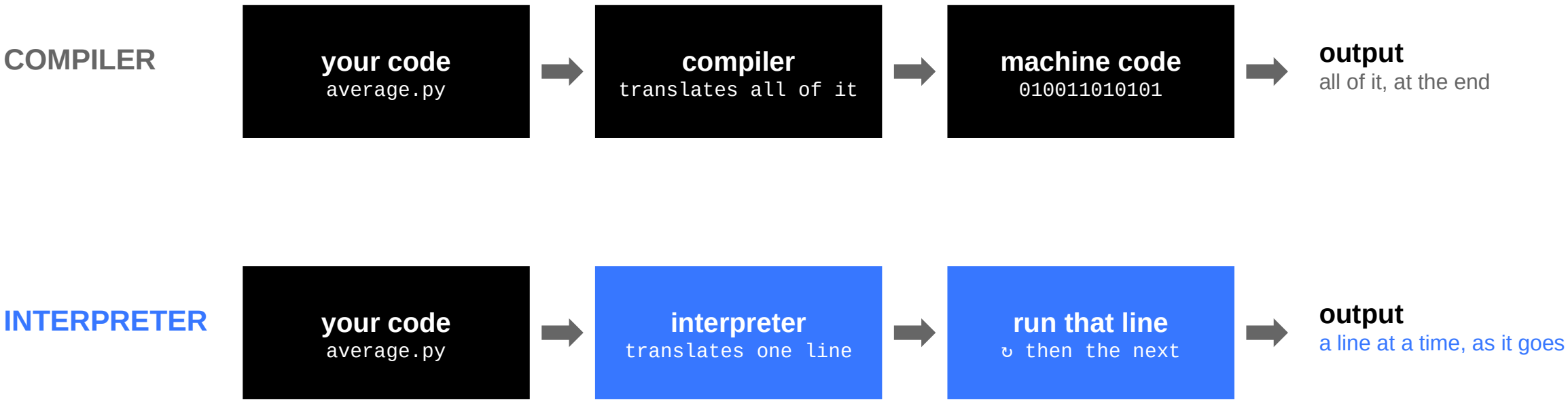
The first program every programmer writes

```
print("Hello, world!")
```

```
>>> Hello, world!
```

Two ways to run a program

Translate it all first, or translate as you go



Python is the second one.

📬 Compiler or interpreter

Two ways to deal with a language you don't speak

A compiler: reads your whole letter***, translates all of it***, and only then hands it over. Nothing runs until everything is understood.

An interpreter: reads your whole letter, ***translates one line, does it***, translates the next. Python works this way.

Which means: in Python, your program *can* print three correct lines and then fail on the fourth. The three lines do not come back.

02.

When the machine tells you

Two kinds of error, and both are cheap

LIVE

Break it on purpose

Delete one character and see what it says

```
print("A")
print("Hello, world!"
```

```
  File "05_hello_broken.py", line 2
    print("Hello, world!"
          ^
SyntaxError: '(' was never closed
```

A syntax error is the cheapest error you will ever have

Nothing has happened yet

The machine refused to run at all. No output, no damage, no wrong answer given to anybody.

The message tells you the file, the line, and often the exact character.

You will see hundreds of these. They are not a judgement on you — they are the machine saying it could not read the sentence.

LIVE

Perfectly legal, and it dies anyway

Nothing was wrong until the moment it ran

```
total = 60
people = 0

print("A")
print("Each pays:", total / people)
```

```
A
ZeroDivisionError: division by zero
```

A runtime error is a cheap error too

What comes before the crash runs

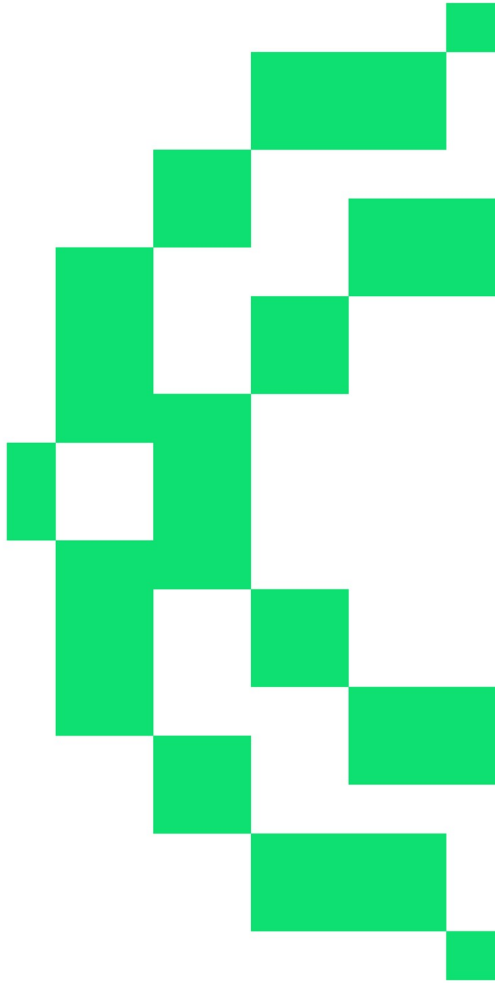
The sentence was legal, so the program started. Then one line asked for something impossible — and it died right there.

The message tells you the line and names the problem: ZeroDivisionError, division by zero. Everything before that line already ran; nothing after it ever will.

It costs more than a syntax error — the program was already running, maybe already answering — but it is still loud. A crash you can see is a crash you can fix.

In both cases, the machine noticed.

It stopped, and it told you where. Remember this — it was the good news.



03.

When the machine says nothing

The bug: code that runs, and quietly does the wrong thing

Back to the final average

Where does the 19 go?

```
grades = [19, 11, 8, 6]

total = 0
for i in range(1, len(grades)):
    total = total + grades[i]

average = total / len(grades)
print("Final average:", average)
```

```
>>> Final average: 6.25
```

A semantic error is the most expensive error you will ever have

Nothing looks wrong at all

The program runs. It produces output. The output is wrong.

No message. No crash. No line number. Nothing at all.

A syntax error costs you thirty seconds. This one can cost you a grade, a client, or a job — and you may never find out it happened.

Three kinds of error

Two of them the machine catches for you

Syntax

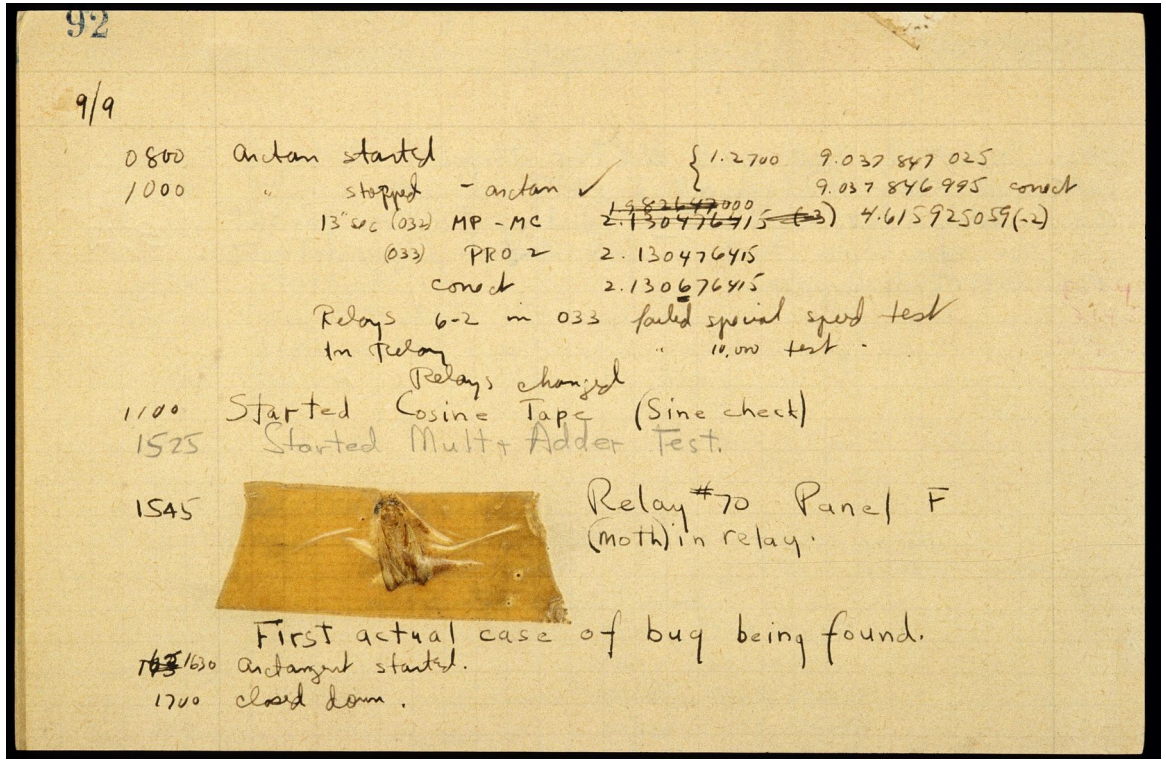
The machine refuses to run · It tells you

Runtime

It starts, then stops · It tells you

Semantic

It runs, and lies · Silence



Smithsonian, NMAH 1994.0191.01

Grace Hopper, again

Harvard, 1947 — the machine stopped, and nobody knew why

They **traced** the fault to Relay #70, Panel F, and found a moth wedged in the contacts. They taped it into the logbook and wrote one line.

“First actual case of bug being found.”

Errors are cheaper when the code is organised

The part you can actually control

You will not stop making mistakes. What you can decide is how long each one takes to find — and that is a question of organisation, not of talent.

You are not writing code for today. You are writing it for whoever has to change it in the future. Usually that person is you — make your own future easier.

Two things do most of the work:

- **comments**, which state what the code is supposed to do, and
- **indentation**, which shows what belongs inside what.

One of them is good manners. In Python, the other one is the law.

Same program, twice

Identical behaviour. Identical bug. One is faster to fix.

```
l = [19, 11, 8, 6]
s = 0
for i in range(1, len(l)):
    s = s + l[i]
print(s / len(l))
```

```
"""Final average of a list of grades."""

# the data
grades = [19, 11, 8, 6]

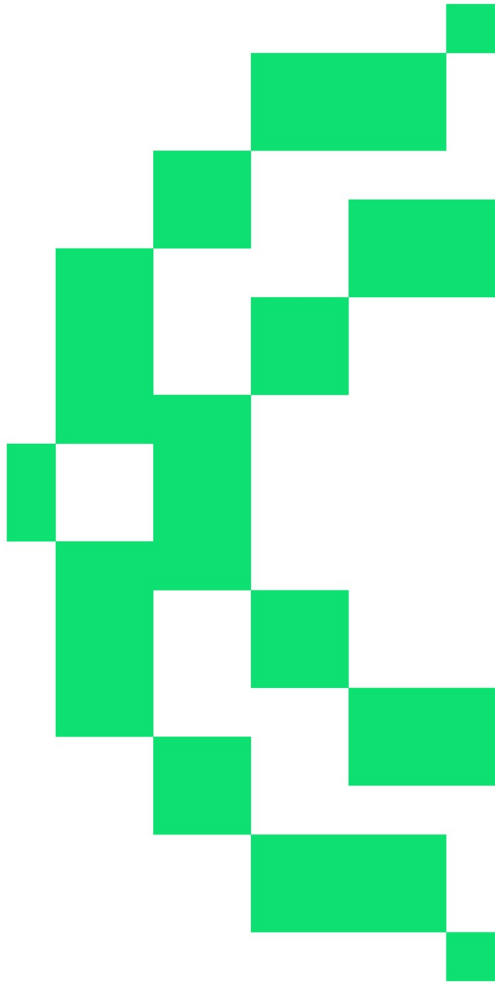
# add up every grade in the list
total = 0
for i in range(1, len(grades)):
    total = total + grades[i]

# the answer
print(total / len(grades))
```

The comment says every grade.

The code does not.

Names and comments are not decoration. They are the claim the code can be checked against — without them there is nothing to compare it to.



One character

range(1, ...) starts counting at the second grade; change to range(0, ...) and the comment is telling the truth

```
# add up every grade in the list
for i in range(0, len(grades)):
    total = total + grades[i]
```

```
>>> Final average: 11.0
```

And in Python, layout is not a matter of taste

Indentation is syntax

In most languages, misaligning your code is ugly. **In Python, alignment changes what the program does.**

The four spaces in front of a line are not there to look neat. They are how the language knows which lines belong inside the loop.

You will meet this properly later in the course. For now: when Python complains about indentation, it is not being fussy.

LIVE

The same code, minus four spaces

Only one of these two is a program

```
"""Final average of a list of grades."""

# the data
grades = [19, 11, 8, 6]

# add up every grade in the list
total = 0
for i in range(0, len(grades)):
    total = total + grades[i]

# the answer
print(total / len(grades))
```

```
"""Final average of a list of grades."""

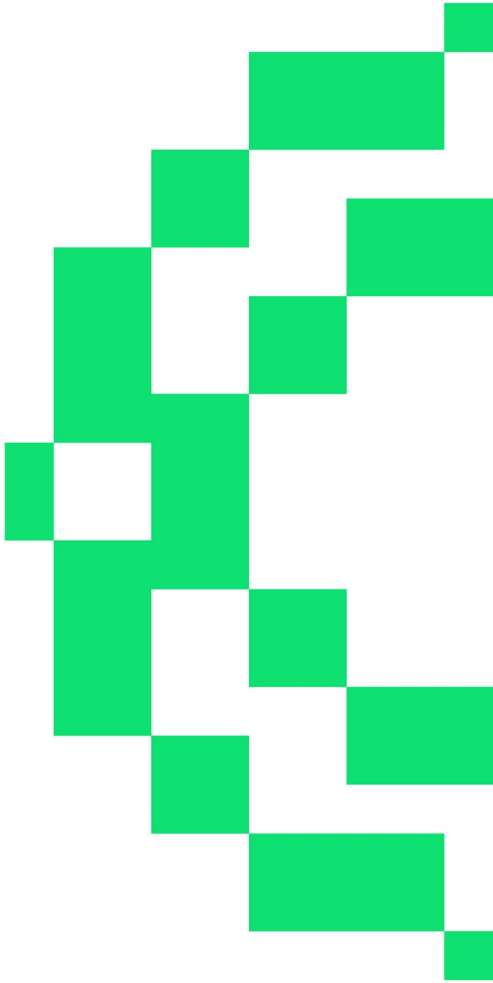
# the data
grades = [19, 11, 8, 6]

# add up every grade in the list
total = 0
for i in range(0, len(grades)):
total = total + grades[i]

# the answer
print(total / len(grades))
```

The tools got very good at producing code that runs.

Nobody has made them good at knowing what you meant. That part didn't get automated — it got more valuable.



Before the next theoretical lecture

On your own

Think Python — Sections 1.6 Formal and natural languages · 1.7 Debugging · 2.8 Comments · 2.9 Debugging.

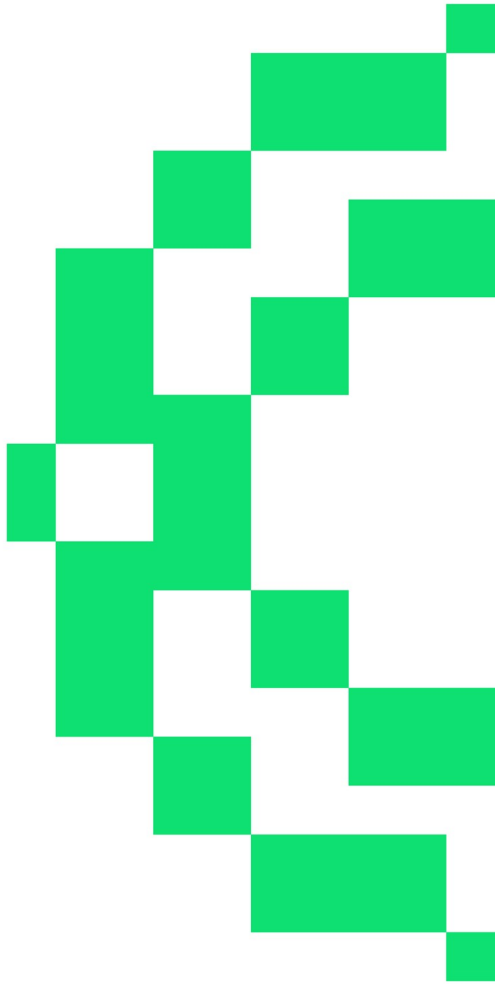
One habit — before you trust a number a program printed, work out by hand what it should have been.

You have finished your first programming class,

and you have already caught something

a machine could not.

That is the job.



NOVA IMS INFORMATION MANAGEMENT SCHOOL

```
input("Questions? ")
```

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · Lapcer · IQNet

Cofinanciado por

PORTUGAL 2030 · 2030 Lisboa Programa Regional de Lisboa · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU

© Karina Rebuli, 2026

One thing to take away and try

Run it before the next lecture. Bring me *your* explanation.

```
print("2" + "2")
```